

Harness-R1: Learning to Edit Executable Runtime Harnesses from Agent Failure Trajectories

Shuai Shao^{1,2†*}, Kangning Zhang^{1,2†*}, Qingyao Li^{1,2*}, Shijian Wang³, Hao Wang², Wenxiang Jiao^{2,✉}, Yuan Lu^{2,✉}, Yi Guo², Weiwen Liu^{1,✉}, Weinan Zhang^{1,✉}

¹Shanghai Jiao Tong University, ²Xiaohongshu Inc., ³Southeast University

*Work done during internship at Xiaohongshu Inc., [†]Equal contribution, [✉]Corresponding authors

Agents built around large language models continually accumulate interaction trajectories during deployment, yet their behavior typically remains fixed. Beyond updating model weights, these trajectories can improve the agent harness that constructs context, mediates tools, validates actions, and recovers execution. We introduce Harness-R1, the first method, to our knowledge, that makes failure-conditioned, lifecycle-wide editing of an existing executable runtime a learned capability. It post-trains a dedicated harness engineer with online reinforcement learning so that its edits are optimized for the realized task success they produce, rather than proposed by a fixed editor. A separate 9B engineer converts batches of target-agent failures into validated executable patches; fresh same-batch reruns of the frozen target provide outcome rewards, so training updates only the engineer. Cold-start supervised fine-tuning initializes this editing policy, which is then trained online with group-relative policy optimization. Across WebShop, ALFWorld, and DBBench, Harness-R1 raises vanilla Qwen3.5-9B success from 44.3% to 53.6% (+9.3 percentage points). After direct target-agent fine-tuning, a target-specific engineer raises the average further from 59.2% to 64.2% (+5.0 points); because these gains hold both before and after fine-tuning the target, Harness-R1 points toward co-evolving the harness engineer and the target agent.

✉ **Contact:** shaoshuai.ederson@sjtu.edu.cn, wenxiangjiaonju@gmail.com, liuww@sjtu.edu.cn

🔗 **Code:** <https://github.com/DeepExperience/Harness-R1>

📦 **Models:** <https://huggingface.co/ShaoShuai0605/Harness-R1>

1 Introduction

Large language models serve as the decision core of tool-using agents, enabling them to interpret tasks, maintain state, and pursue complex goals through multi-turn interaction with external environments (Yao et al., 2023b; Liang et al., 2024; Li et al., 2026a; Zhang et al., 2025). Unlike a single model invocation, a deployed agent continually produces trajectories containing observations, actions, environment feedback, and task outcomes. These trajectories record successful experience, but they also expose systematic failures such as tool misuse, lost state, protocol violations, repeated attempts, and failed recovery. This raises a natural question: can agents use their interaction experience to improve continually rather than remain fixed after deployment? This experience-to-improvement loop is a central concern of self-evolving agents (Gao et al., 2026; Wu et al., 2026; Yu et al., 2026).

An agent system can improve at two complementary locations. One line updates model parameters through supervised fine-tuning, reinforcement learning, or online learning, directly improving the actor that makes task decisions (Xia et al., 2026; Lu et al., 2026; Shi et al., 2026). The other keeps the model fixed and optimizes the *agent harness* around it. Context construction, memory and skills, tool mediation, action validation, and control and recovery logic are all harness components (Shinn et al., 2023; Zhao et al., 2024; Karten et al., 2026). Together, they determine what the model observes, which actions it can execute, how it interprets environment feedback, and how execution recovers after deviations. Identical model weights can therefore yield substantially different agent capabilities under different harnesses. Harness optimization offers a complementary path to model training: it improves the runtime mechanisms between a model and its environment without changing the model itself.

Direct harness modification is not uniformly reliable. Figure 1 compares matched-baseline changes in mean envi-

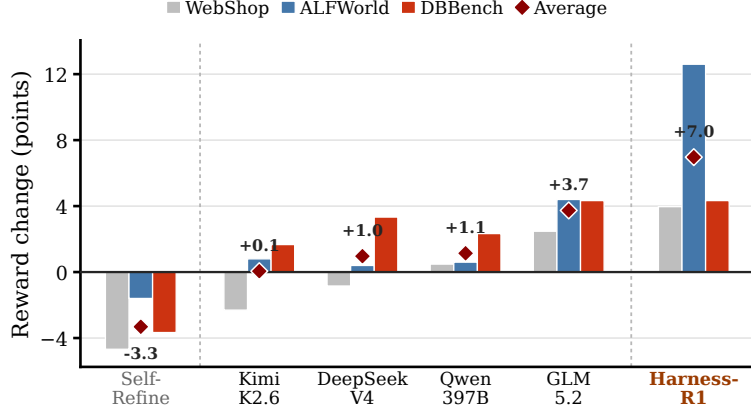


Figure 1 Matched-baseline reward changes across three benchmarks; diamonds denote the equal-weight average.

ronment reward across the three benchmarks. A fixed Self-Refine rule (Madaan et al., 2023) lowers reward on all three benchmarks, and the gains from frontier harness editors are unstable or limited, with some even reducing the WebShop reward. Prompting strong but fixed models to edit the harness is therefore not reliable enough.

Beyond such prompted edits, recent systems build dedicated harness-optimization pipelines. Meta-Harness, Agentic Harness Engineering, and AutoHarness use agentic proposers to jointly edit prompts, tools, memory, middleware, or control logic from harness state, execution traces, and task feedback; Life-Harness and HarnessX extend the editable surface to lifecycle interventions and typed components (Lee et al., 2026; Lin et al., 2026; Lou et al., 2026; Xu et al., 2026; Chen et al., 2026b). Yet the harness proposer usually remains fixed: outcomes select or iteratively refine patches without directly updating proposer parameters; HarnessX uses cross-harness GRPO to train the task model, while AEGIS retains symbolic harness editing. Complementary work optimizes prompts, demonstrations, memories, skills, or task-solving programs (Yang et al., 2024; Khattab et al., 2023; Shinn et al., 2023; Zhao et al., 2024; Xia et al., 2026; Lu et al., 2026; Shi et al., 2026), but typically isolates one artifact or constructs a new solution program. A workflow may be part of a harness, but generating one differs from learning to install failure-conditioned executable interventions into an existing multi-stage runtime. The latter must decide when to intervene from actual target-agent failures and coordinate context, state, action execution, and recovery. This leaves a less studied question: can we post-train a dedicated harness engineer with online reinforcement learning, so that improving an existing executable runtime from observed failures becomes a learned capability?

Training a harness engineer directly poses two challenges. First, the editable runtime spans interdependent execution stages, so unrestricted code edits can break existing interfaces, produce non-executable behavior, or introduce changes unrelated to task success. Second, text form and static rules cannot determine patch quality; only the target agent’s behavior after applying the patch can do so. Training therefore requires a grounded feedback path from target-agent failures, through constrained executable edits, to the performance gain that updates the engineer.

We introduce Harness-R1, a training paradigm that post-trains a dedicated harness engineer with online reinforcement learning while keeping the target agent frozen. Conditioned on batches of target-agent failures, the engineer generates validated executable runtime patches; the patched target reruns the same tasks, and the realized performance change rewards only the engineer. Cold-start supervised fine-tuning initializes the policy before online GRPO. Across WebShop, ALFWorld, and DBBench, Harness-R1 raises average success from 44.3% to 53.6% (+9.3 points) for the vanilla target and from 59.2% to 64.2% (+5.0 points) after direct target-agent fine-tuning.

The main contributions can be summarized as follows.

- We formulate failure-conditioned, lifecycle-wide harness editing as an online reinforcement-learning problem for a dedicated engineer while keeping the target agent frozen.
- We develop Harness-R1, combining cold-start supervised fine-tuning with group-relative policy optimization over the realized utility of executable runtime patches, and it improves the vanilla target by 9.3 points across three interactive benchmarks.
- We show that the harness engineer and the target agent can co-evolve: after direct target-agent fine-tuning, a

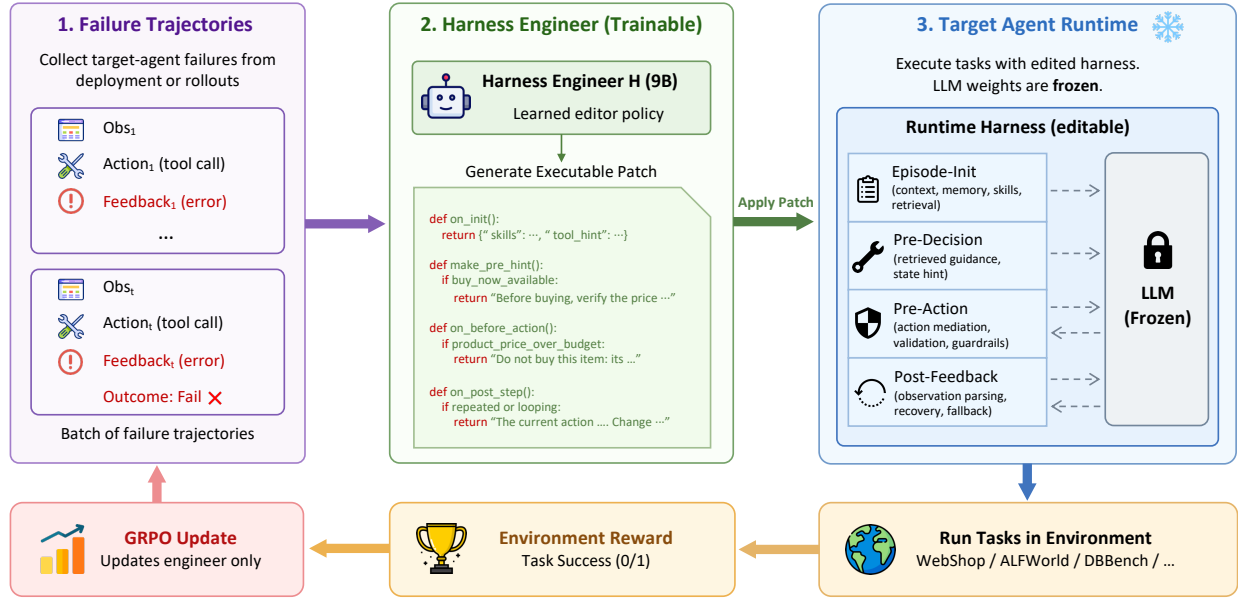


Figure 2 Overview of Harness-R1. Mined target failures become an evidence bundle (left); the harness engineer writes an executable patch that edits four lifecycle points in the runtime surrounding the frozen target (right); the patched target reruns the same tasks, and the resulting same-batch reward trains only the engineer via cold-start SFT and then GRPO (bottom loop). See the Method section for the loop, action space, and lifecycle hooks.

target-specific Harness-R1 engineer adds a further 5.0 points.

2 Related Work

LLM-Based Harness Evolution. Recent work treats the agent harness as an executable, multi-component optimization object. One line searches or synthesizes whole harnesses from execution traces, evaluation scores, and rewards: Meta-Harness has a coding agent search over prior candidates, AutoHarness iteratively synthesizes code harnesses from environment-validity feedback, and AHE jointly evolves prompts, tools, middleware, skills, sub-agents, and memory (Lee et al., 2026; Lou et al., 2026; Lin et al., 2026). A second line turns recurring interaction failures into scoped, regression-checked repairs across multiple execution stages, combining trace-grounded diagnosis with regression-aware validation (Xu et al., 2026; Chen et al., 2026a; Zhang et al., 2026a). As a closely related concurrent direction, HarnessX composes typed processors, performs symbolic trace-driven adaptation with AEGIS, and applies cross-harness GRPO to the task model (Chen et al., 2026b). Across these systems the proposer may be a stronger external model or the target model itself, but none post-trains the proposer or editor weights from harness-editing outcomes; feedback instead guides program search, candidate selection, regression testing, or artifact promotion. Harness-R1 instead moves the learning target from the resulting harness to the editing policy: online reinforcement learning post-trains a dedicated harness engineer from the realized utility of its patches on a frozen target agent.

Algorithmic Optimization of Harness Components. A broader line optimizes prompts and other model-external artifacts within prespecified edit spaces and search procedures. Some methods have an LLM propose and search over instruction candidates against a score, as in APE and OPRO (Zhou et al., 2023; Yang et al., 2024). Others form natural-language “gradients” or textual feedback and propagate them to edit prompts or code, as in ProTeGi and TextGrad (Pryzant et al., 2023; Yuksekogonul et al., 2025). Population-based methods such as EvoPrompt, Promptbreeder, and GEPA mutate and select prompts by fitness (Guo et al., 2025; Fernando et al., 2023; Agrawal et al., 2026), whereas pipeline compilers such as DSPy and MIPRO separate program structure from module parameters and search over instructions and bootstrapped demonstrations (Khattab et al., 2023; Opsahl-Ong et al., 2024). These methods can invoke strong language models and maintain histories, populations, or learned surrogates, but they generally do not post-train the proposer, reflector, or backward engine from editing outcomes; even when some fine-tune task modules,

the product is a task-specific artifact or parameter rather than a dedicated harness-editor policy trained by editing outcomes. Harness-R1 instead trains the editor from the realized execution effects of its patches and modifies multiple stages of an existing target-agent runtime.

Learned Harness Editors. More directly related work post-trains policies that edit or control model-external structures, but each either restricts the edit space or couples editing with task solving. Some train a dedicated editor from downstream outcomes over a narrow target, such as an independent context field or a revisable skill bank (Chen et al., 2026c; Vishe et al., 2026; Li et al., 2026b), or learn to generate task-solving workflows run by a frozen executor (Li et al., 2024; Nie et al., 2025; Zhang et al., 2026b). Closer to the runtime harness, others instruction-tune observation and action projections without reinforcement learning or persistent code patches, select among a few predefined structural actions under offline RL, or fold harness edits into a single task actor’s action space (Wang et al., 2026; Yi and Song, 2026; Luo et al., 2026). In contrast, Harness-R1 isolates harness editing as a learning problem in its own right, casting failure-conditioned, lifecycle-wide editing as a standalone online reinforcement-learning task for a dedicated engineer, trained from the realized task outcomes of its executable patches while the target agent stays frozen.

3 Method

In this section, we introduce Harness-R1, an online, outcome-grounded framework that post-trains a dedicated engineer to improve the executable runtime surrounding a frozen target agent. We first formulate harness editing as a batch-conditioned learning problem in which each modification is evaluated by rerunning the same tasks. We then describe where the modification can intervene across the agent lifecycle and how cold-start supervised fine-tuning followed by GRPO learns the editing policy. Figure 2 summarizes this failure-to-edit-to-rerun loop and the separation between the trainable engineer and the frozen task agent.

3.1 Problem Setup

Let A denote a frozen target agent (its model together with the surrounding base runtime), and let $B = \{x_i\}_{i=1}^n$ be a batch of n tasks in environment E . Within an episode the agent interacts with the environment over multiple turns: at each turn it reads the accumulated history and the current observation, and its frozen policy proposes an action. Actions are expressed in the environment’s native interface: structured tool or API calls and textual commands, such as search and click in WebShop, navigation and object manipulation in ALFWorld, and SQL queries in DBBench. The environment executes the action, returns the next observation, and emits an outcome reward once the episode ends. The component we adapt is the base runtime: the code that assembles the context shown to the model, forwards each action to the environment, and relays the feedback back to the agent. This surrounding runtime, and not the model’s weights, is exactly what the harness edits. Running the unmodified agent over the batch yields baseline trajectories τ_i^0 and rewards R_i^0 . A deterministic extractor retains only failed episodes and compacts their task constraints, selected action–observation excerpts, outcomes, and necessary environment state into a failure packet s_B . The engineer H_θ reads this packet once and generates a batch-conditioned executable overlay P ; it neither answers the tasks nor participates in their rollouts.

The overlay P wraps this loop as executable hooks at four lifecycle points, leaving the agent’s weights untouched (Figure 2, right): (i) *episode initialization* sets up the starting context and episode state; (ii) *pre-decision* augments the context with retrieved guidance and interface constraints before the agent decides; (iii) *pre-action* is a runtime guardrail that may canonicalize, rewrite, or veto the proposed action before it reaches the environment; and (iv) *post-feedback* inspects the returned observation and triggers recovery when the trajectory stalls. These hooks thus touch only the inputs and outputs surrounding the frozen policy, never the policy itself, and patches are validated before installation, with invalid patches having no effect. Appendix C gives the invocation point and the permitted return effect of each hook.

After validation, the overlay is installed and the same frozen target reruns every task in B , including tasks that originally succeeded. Let R_i^P denote the resulting reward. Define the full-batch performance difference and the

engineer reward as

$$\begin{aligned}\Delta_B(P) &= \frac{1}{n} \sum_{i=1}^n (R_i^P - R_i^0), \\ r(B, P) &= \begin{cases} \Delta_B(P), & \text{if valid and complete,} \\ 0, & \text{otherwise.} \end{cases}\end{aligned}\tag{1}$$

Using the same tasks before and after editing controls task composition but defines a same-batch, transductive objective, with no iterative refinement within an instance or persistent patch memory across batches. This reward is non-differentiable and observable only after an edit changes target-agent behavior, so it cannot be optimized directly.

3.2 Outcome-Grounded Post-Training

Harness-R1 learns the editing policy in two stages. Cold-start supervised fine-tuning first initializes a prior over valid, executable edits, and online, outcome-grounded GRPO then optimizes the realized task utility of patches applied to the frozen target.

Cold-start supervised fine-tuning. We first run the frozen target with its base harness and form editing instances from the resulting failed trajectories. The teacher and RL instances use disjoint task batches. A strong teacher proposes serialized editing responses y_j^T from the compact failure packets s_j ; we validate and evaluate their parsed overlays with the frozen target, retaining at most one executable, complete, non-regressive response per packet. The resulting dataset $\mathcal{D}_{\text{SFT}} = \{(s_j, y_j^T)\}_{j=1}^M$ initializes the engineer by teacher-forced next-token prediction:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{\sum_{j=1}^M |y_j^T|} \sum_{j=1}^M \sum_{t=1}^{|y_j^T|} \log H_{\theta}(y_{j,t}^T \mid s_j, y_{j,<t}^T).\tag{2}$$

Outcome-grounded GRPO. Starting from the supervised policy, we perform online GRPO (Shao et al., 2024) and sample $K = 8$ candidate patches from the current policy for each failure packet (Alg. 1, lines 4–5). Each candidate is parsed and validated into a patch (line 6); each valid patch is then installed independently and evaluated by rerunning the frozen target on the same full task batch (line 7), while invalid, no-op, or incomplete evaluations receive zero reward under Eq. (1) (line 8). For rewards $r_k = r(B, P_k)$, let μ_B and σ_B be the empirical mean and standard deviation within the eight candidates generated from the same packet, and normalize the rewards into advantages (line 10):

$$\hat{A}_k = \frac{r_k - \mu_B}{\sigma_B}.\tag{3}$$

Let $y_k = (y_{k,1}, \dots, y_{k,T_k})$ be the engineer response parsed into P_k , and let $\rho_{k,t}(\theta) = H_{\theta}(y_{k,t} \mid s_B, y_{k,<t}) / H_{\theta_{\text{old}}}(y_{k,t} \mid s_B, y_{k,<t})$. The sequence-level advantage is shared by all response tokens, and the engineer maximizes the token-averaged clipped surrogate

$$\begin{aligned}g_{k,t}(\theta) &= \min\left\{\rho_{k,t}\hat{A}_k, \right. \\ &\quad \left.\text{clip}(\rho_{k,t}, 1 - \epsilon_{\ell}, 1 + \epsilon_h)\hat{A}_k\right\}, \\ \mathcal{J}(\theta) &= \mathbb{E}\left[\frac{1}{K} \sum_{k=1}^K \frac{1}{T_k} \sum_{t=1}^{T_k} w_{k,t} g_{k,t}(\theta)\right].\end{aligned}\tag{4}$$

Here $\ell_{k,t}^{\text{tr}}$ and $\ell_{k,t}^{\text{ro}}$ are the old-policy token log probabilities recomputed by the training engine and recorded by the rollout engine, respectively; $w_{k,t} = \text{clip}(\exp(\ell_{k,t}^{\text{tr}} - \ell_{k,t}^{\text{ro}}), 0, 2)$ is the truncated importance weight. WebShop supplies shaped environment reward, whereas ALFWorld and DBBench supply binary success; no format-validity bonus or explicit KL loss is added. Only the engineer parameters θ are updated (line 12), and the outer loop iterates over update bundles until the training budget is exhausted (lines 2 and 13).

Algorithm 1 summarizes the online RL stage. Base trajectories, rewards, and failure packets are cached before optimization; online evaluation reruns only the patched target for candidates sampled from the current engineer.

Algorithm 1 Online RL for harness editing.

Input: Frozen target A , cached RL records $\mathcal{Q}_{\text{RL}} = \{(B, s_B, \mathbf{R}_B^0)\}$, initialized engineer H_θ

Parameters: group size K , clipping bounds $\epsilon_\ell, \epsilon_h$, learning rate η , update-bundle size $|\mathcal{U}|$, training budget

Output: Trained harness engineer H_{θ^*}

```
1: repeat
2:   Sample an update bundle  $\mathcal{U} \subset \mathcal{Q}_{\text{RL}}$ ; set  $\theta_{\text{old}} \leftarrow \theta$ .
3:   for each  $(B, s_B, \mathbf{R}_B^0) \in \mathcal{U}$  do
4:     Sample  $\{y_k\}_{k=1}^K \sim H_{\theta_{\text{old}}}(\cdot \mid s_B)$ .
5:     for  $k = 1, \dots, K$  do
6:       Parse and validate  $y_k$  into patch  $P_k$ .
7:       Independently install valid  $P_k$ , rerun frozen  $A$  on all tasks in  $B$ , and compute  $r_k$  with Eq. (1).
8:       Set  $r_k = 0$  for invalid, no-op, or ultimately incomplete evaluations.
9:     end for
10:    Normalize  $\{r_k\}_{k=1}^K$  into  $\{\hat{A}_k\}$  with Eq. (3).
11:  end for
12:  Update only  $\theta$  with Eq. (4).
13: until the training budget is exhausted; return  $H_\theta$ .
```

4 Experiments

We evaluate Harness-R1 on three interactive environments that stress different forms of agent execution. We ask whether outcome-trained harness editing improves over supervised editing and strong fixed editors, remains useful after direct target-agent training, transfers to unseen target models and tasks, and which lifecycle positions drive the gains.

4.1 Experimental Setup

Benchmarks. WebShop (Yao et al., 2023a) evaluates grounded web navigation: an agent must search, inspect, and purchase a product satisfying a natural-language request. ALFWorld (Shridhar et al., 2021) is a text-based embodied environment whose household tasks require multi-step navigation, object manipulation, state tracking, and recovery. DBBench from AgentBench (Liu et al., 2025) is a relational-database environment whose natural-language tasks require schema inspection, structured SQL querying, record manipulation, and result verification. Together, the three environments expose complementary failures in long-horizon interaction, action execution, and interface compliance.

Target agents and comparisons. Our primary target is a frozen Qwen3.5-9B agent (Qwen Team, 2026). To test whether harness adaptation remains useful after improving the actor itself, we also evaluate the same backbone after direct task-agent SFT. Beyond this primary target, we further probe cross-model transfer by applying the trained engineer to a broad set of target agents unseen during training. We compare the unmodified target against four groups: fixed prompt-based agentic strategies (ReAct (Yao et al., 2023b), Self-Refine (Madaan et al., 2023), and Reflection (Shinn et al., 2023)); strong frontier models prompted as harness engineers (Qwen3.5-397B, GLM-5.2, Kimi-K2.6, DeepSeek-V4-Pro, Gemini-3.5-Flash, and GPT-5.5) (Qwen Team, 2026; Z.ai, 2026; Moonshot AI, 2026; DeepSeek-AI et al., 2026; Google DeepMind, 2026; OpenAI, 2026); a supervised-only engineer; and outcome-trained Harness-R1. Within each benchmark, an editor is evaluated against the same target and task set without its generated patch.

Evaluation. We report task success on all three benchmarks and the shaped environment score on WebShop; success is computed over 500, 500, and 300 tasks for WebShop, ALFWorld, and DBBench, respectively. ALFWorld additionally reports success across its six task families and a task-level micro-average (All), and we report the average across the three benchmarks (Avg.) as the overall summary. Reflection is reported under a separate two-episode success@2 protocol: its success columns are cumulative over two episodes and its Score is measured after retrying first-episode failures, whereas all other rows report success@1. It is therefore not ranked against single-episode methods.

Training and selection. The engineer is a separate 9B model initialized by cold-start SFT and then optimized with online GRPO while the target remains frozen. The cold-start SFT set comprises roughly 1,000 executable editing examples proposed by a GPT-5.5 teacher and filtered by validation on the frozen target, and online GRPO trains

Method	ALFWorld							WebShop		DBBench	Avg.
	Pick	Look	Clean	Heat	Cool	Pick2	All	Score	Succ.	Succ.	
Qwen3.5-9B (with default harness)	75.8	66.7	16.9	28.8	9.2	47.5	40.6	66.0	31.2	61.0	44.3
Prompt-based Agentic Methods											
ReAct	79.8	54.8	27.3	27.4	10.3	53.3	43.4	66.8	37.4	61.7	47.5
Self-Refine	70.7	45.2	11.7	28.8	6.9	57.4	39.0	61.7	29.0	57.3	41.8
Reflection	91.9	90.5	24.7	56.2	19.5	73.8	59.2	61.2	43.6	64.7	55.8
Frontier Models											
Qwen3.5-397B	76.8	66.7	14.3	31.5	11.5	47.5	41.2	66.4	32.8	63.3	45.8
GLM-5.2	77.8	78.6	18.2	27.4	16.1	54.9	45.0	68.4	36.0	65.3	48.8
Kimi-K2.6	73.7	71.4	14.3	30.1	12.6	49.2	41.4	63.7	31.8	62.7	45.3
DeepSeek-V4-Pro	72.7	69.0	13.0	30.1	16.1	47.5	41.0	65.1	32.4	64.3	45.9
Gemini-3.5-Flash	67.7	69.0	14.3	24.7	10.3	35.2	35.4	63.2	33.6	64.0	44.3
GPT-5.5	80.8	78.6	31.2	28.8	17.2	35.2	43.2	61.4	36.6	64.0	47.9
Ours											
Supervised-only engineer	80.8	66.7	22.1	24.7	11.5	36.1	39.4	67.8	38.6	61.3	46.4
Harness-R1	77.8	81.0	58.4	43.8	34.5	39.3	53.2	69.9	42.2	65.3	53.6
Agent SFT	91.9	100.0	46.8	56.2	48.3	85.2	71.2	71.5	42.6	63.7	59.2
Agent SFT + Harness-R1	93.9	100.0	81.8	74.0	72.4	86.1	84.0	68.7	43.0	65.7	64.2

Table 1 Main results across WebShop, ALFWorld, and DBBench (%). Score is the mean shaped reward and Succ. is the task success rate. Avg. is the equal-weight average of WebShop Succ., ALFWorld All, and DBBench Succ. Across all non-Reflection rows, red and blue mark the highest and second-highest distinct value in each column, respectively; ties share a color.

on roughly 1,500 failure packets from a disjoint task split. We select checkpoints using aggregate development performance and use the same executable patch interface for all trained variants. Appendix A reproduces the engineer prompt template, Appendix B lists the training, decoding, and reward hyperparameters, and Appendix D details the task-level SFT, RL, validation, and test splits.

4.2 Main Results

Table 1 summarizes target-specific performance against prompt-based strategies, frontier engineers, and supervised engineer training. We focus on task success as the primary metric.

Outcome-trained harness editing improves the frozen target. Harness-R1 raises success on all three benchmarks and improves the equal-weight average from 44.3% to 53.6%, a gain of 9.3 percentage points. The largest absolute gain is on ALFWorld, where success rises from 40.6% to 53.2%, while WebShop and DBBench also improve. The outcome-trained engineer is 7.1 points above the supervised-only engineer.

A dedicated trained engineer is more effective than fixed alternatives. Among frontier engineers, the strongest is GLM-5.2 at a 48.8% average, below Harness-R1 at 53.6%. Prompt-based strategies are not uniformly beneficial: ReAct improves the average by 3.2 points, whereas Self-Refine reduces it by 2.5 points. Reflection reaches 55.8% cumulative success under its two-episode protocol, which is not directly comparable to the single-episode rows.

The harness engineer co-evolves with the target agent. Direct agent SFT raises the unmodified target to a 59.2% average, and a target-specific Harness-R1 engineer trained for this stronger actor raises it further to 64.2%, an additional 5.0 points. Harness editing therefore keeps improving the target even after the actor itself has been fine-tuned, showing that the engineer can co-evolve with the target agent rather than saturating once the agent improves. The gain concentrates in task success, especially on ALFWorld; although a few individual metrics dip slightly, Harness-R1 still improves the overall success of the fine-tuned agent.

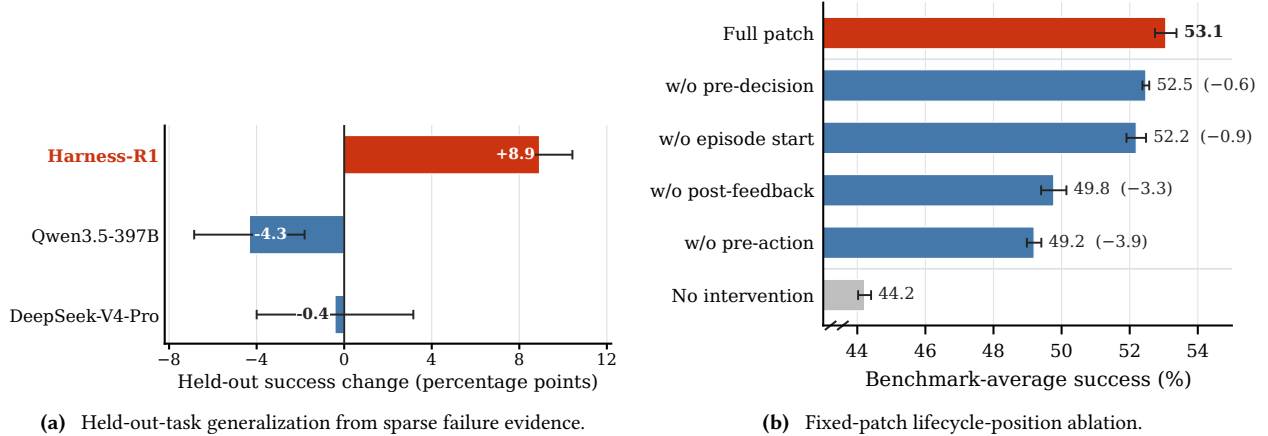


Figure 4 Analysis of Harness-R1 on the vanilla Qwen3.5-9B target. (a) Held-out-task generalization from sparse failure evidence: bars show mean pooled success-rate change over three matched evidence seeds, and whiskers show sample standard deviation. (b) Fixed-patch lifecycle-position ablation: success is benchmark-averaged and the horizontal axis is truncated.

4.3 Generalization across Target Agents

We next ask whether the learned editing policy can adapt to target models unseen during training. Each target supplies its own failure traces and receives a newly generated patch, so this experiment tests editor-policy transfer rather than replaying a fixed patch. Across twenty unseen target configurations, the benchmark-averaged gain is 7.06 percentage points, and every target-level average is positive. Across the full 21-target matrix, 56 of 63 target–benchmark combinations improve, four are unchanged, and the three regressions are all small (≤ 2.0 points; Figure 3). Aggregating matched tasks within each benchmark, gains stay positive at 4.15 points on WebShop, 9.63 on ALFWORLD, and 7.37 on DBBench, with every delta computed on a matched target-specific task set. The learned editing policy thus generalizes strongly: a single training recipe transfers to targets of different families and scales, improving every one without any per-target retuning. Appendix E reports the per-target success rates before and after patch installation that underlie these deltas.

4.4 Held-Out Task Generalization

We test whether sparse failures yield patches that improve unseen tasks. For each benchmark and seed, every engineer observes the same 10 failures from the frozen Qwen3.5-9B target, generates one benchmark-specific patch, and applies it to all other tasks. The pooled held-out set contains 1,270 tasks across WebShop, ALFWORLD, and DBBench; we repeat the protocol over three matched seeds.

Figure 4a shows that Harness-R1 improves pooled held-out success by 8.9 ± 1.5 percentage points and is positive for all three seeds. Under the same protocol, Qwen3.5-397B and DeepSeek-V4-Pro yield -4.3 ± 2.5 and -0.4 ± 3.6 points, respectively. The gap is not only in the mean: both frontier engineers straddle zero across seeds (spreads of ± 2.5 and ± 3.6 points around negative averages), swinging between marginal gains and sizable regressions, whereas Harness-R1 stays positive on every seed at a tighter ± 1.5 . Converting a handful of failures into a broadly useful edit is thus a capability that scale alone does not confer, and one that outcome-grounded training makes both stronger and more consistent. Appendix F additionally reports how many seed-benchmark patches installed a real intervention and the corresponding full-split changes.

	WebShop	ALFWORLD	DBBench	Avg.
Llama-3.1-8B	+9.8	+8.0	+13.7	+10.5
Llama-3.1-70B	-0.4	+14.2	+2.0	+5.3
Llama-3.2-1B	+0.0	+0.2	+6.3	+2.2
Llama-3.2-3B	+5.2	+6.0	+9.7	+7.0
Llama-3.3-70B	+6.4	+18.8	+2.7	+9.3
Gemma-3-1B	+0.0	+1.2	+2.3	+1.2
Gemma-3-4B	+14.6	+6.4	+15.0	+12.0
Gemma-3-12B	+9.4	+6.0	+15.7	+10.4
Gemma-3-27B	+9.4	+10.8	+7.7	+9.3
Gemma-4-12B-it	+0.0	+19.4	+6.3	+8.6
Gemma-4-26B-A4B-it	+0.4	+16.2	+9.0	+8.5
Gemma-4-31B-it	-0.2	+18.8	+3.3	+7.3
Qwen2.5-72B	+1.8	-2.0	+3.3	+1.0
Qwen3-4B	+13.6	+6.3	+17.0	+12.3
Qwen3-8B	+4.0	+4.7	+8.0	+5.6
Qwen3-14B	+3.4	+16.7	+10.0	+10.0
Qwen3.5-4B	+3.6	+17.9	+4.3	+8.6
Qwen3.5-9B	+11.0	+12.6	+4.3	+9.3
Qwen3.5-27B	+0.0	+8.9	+2.0	+3.6
Qwen3.5-35B-A3B	+1.2	+7.0	+6.0	+4.7
Qwen3.6-27B	+0.8	+8.0	+3.0	+3.9

Figure 3 Target-agent generalization in success-rate points; Avg. weights benchmarks equally.

4.5 Where in the Lifecycle Do Modifications Matter?

Which intervention points account for the improvement? Figure 4b holds the frozen target and generated patches fixed, then disables one lifecycle position at a time alongside no-intervention and full-patch controls. The no-intervention, full-patch, and leave-one-position-out arms each rerun the target three times per benchmark and use the same equal-benchmark weighting as Table 1. Whiskers are standard deviations across benchmark recombinations. The full patch reaches 53.1% average success, 8.9 points above no intervention. Removing pre-action mediation or post-feedback recovery reduces success by 3.9 and 3.3 points, whereas removing episode-start or pre-decision changes costs 0.9 and 0.6 points. The dominant position depends on the environment: pre-action mediation matters most on WebShop, while post-feedback recovery matters most on ALFWorld. Because a patch can coordinate multiple positions, and the evaluated WebShop patches contain only pre-action edits, these effects are conditional and should not be added into a universal importance ranking. Appendix G tabulates the per-benchmark success rates behind this ablation.

5 Discussion

As agents begin to improve other agents, harness editing becomes a form of AI improving AI. In this setting, producing edits that merely look correct is not enough: an edit intervenes directly in a running executable system, so it must be precise, verifiable, and genuinely beneficial to the agent it modifies. This is why Harness-R1 learns from the realized task outcome of each patch rather than from whether its text appears reasonable.

Training a dedicated engineer beats prompting a larger model. As shown in the introduction, prompting strong but fixed frontier models to edit the harness is unreliable: they optimize for plausibility, emitting syntactically valid and reasonable-looking edits, but because they never rerun the target they cannot tell whether an edit actually raises task success, so their gains are unstable and sometimes even lower reward. Harness-R1 instead trains on the realized rerun outcome of each patch and learns edits that are genuinely useful rather than merely plausible: a valid, well-formed patch is necessary but not sufficient, and what ultimately matters is whether rerunning the target confirms a task gain. Because the signal comes from outcomes rather than model scale, a 9B engineer trained this way surpasses much larger frontier editors (GLM-5.2 at 48.8% versus Harness-R1 at 53.6%). Appendix H inspects stored patches together with their runtime traces, including a frontier-editor patch whose plausible diagnosis compiles into behavior that lowers success.

Failure-conditioned learning beats fixed harness patterns. Fixed, hand-designed harness strategies such as Self-Refine, Reflection, and ReAct are often assumed to help agents in general, yet they apply one hand-crafted rule uniformly to every task, ignoring both the specific target’s failure modes and whether a given intervention actually works. Our results show this is not reliable: a fixed Self-Refine rule lowers reward on all three benchmarks (an average of 2.6 points), and ReAct yields only limited and inconsistent gains. Harness-R1 instead generates edits conditioned on the target agent’s observed failures and keeps only patches verified to help by rerun, so its improvements adapt to each target rather than betting on a single universal recipe. This adaptivity also appears across the lifecycle: the dominant intervention point varies by environment (pre-action mediation on WebShop, post-feedback recovery on ALFWorld), which a fixed strategy cannot select on its own.

Limitations and future work. In this work, we study a single adaptation from a vanilla target to a fine-tuned one, where a re-trained engineer still improves the stronger actor. A natural extension is to iterate this into multi-round co-evolution that alternates updates to the target agent and the harness engineer, so that gains in one continually reshape the training signal for the other; how such alternation converges and whether it yields compounding gains is a promising path toward agents that keep improving after deployment. Our reward is also computed from same-batch task outcomes, which keeps training grounded but ties the signal to the tasks used to mine failures. Future work can enrich this reward with held-out performance, so that edits are explicitly optimized against regressions on unseen tasks, and with inference-efficiency terms, so that useful patches are not obtained at unnecessary runtime cost, letting a single engineer jointly balance utility, robustness, and cost.

6 Conclusion

In this paper, we formalize failure-conditioned, lifecycle-wide editing of an executable agent harness as an online reinforcement learning problem for a dedicated engineer, while keeping the target agent frozen. We propose Harness-R1, which initializes this editing policy with cold-start supervised fine-tuning and then trains it online with GRPO, so that edits are optimized for the realized task utility of executable runtime patches rather than produced by a fixed editor. Across WebShop, ALFWorld, and DBBench, Harness-R1 improves every benchmark and raises the average success of the vanilla Qwen3.5-9B target from 44.3% to 53.6% (+9.3 points), while an engineer retrained for a directly fine-tuned target further raises it from 59.2% to 64.2% (+5.0 points). The learned editor also generalizes: it transfers to twenty unseen target models with a positive gain on every one and improves 1,270 held-out tasks. Together, these results show that harness construction is a learnable capability that complements weight updates and lets the engineer and target co-evolve.

References

- Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. Gepa: Reflective prompt evolution can outperform reinforcement learning, 2026. URL <https://arxiv.org/abs/2507.19457>.
- Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. From failed trajectories to reliable llm agents: Diagnosing and repairing harness flaws, 2026a. URL <https://arxiv.org/abs/2606.06324>.
- Tingyang Chen, Shuo Lu, Kang Zhao, Weicheng Meng, Hanlin Teng, Tianhao Li, Chao Li, Xule Liu, Jian Liang, Zhizhong Zhang, Yuan Xie, Heng Qu, Kun Shao, and Jian Luan. Harnessx: A composable, adaptive, and evolvable agent harness foundry, 2026b. URL <https://arxiv.org/abs/2606.14249>.
- Xiaoyin Chen, Canwen Xu, Yite Wang, Boyi Liu, Zhewei Yao, and Yuxiong He. Learning to self-evolve, 2026c. URL <https://arxiv.org/abs/2603.18620>.
- DeepSeek-AI et al. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026. URL <https://arxiv.org/abs/2606.19348>.
- Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution, 2023. URL <https://arxiv.org/abs/2309.16797>.
- Huan-ang Gao et al. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *Transactions on Machine Learning Research*, 2026. doi: 10.48550/arXiv.2507.21046. URL <https://arxiv.org/abs/2507.21046>.
- Google DeepMind. Gemini 3.5 Flash: Model card. <https://deepmind.google/models/model-cards/gemini-3-5-flash/>, May 2026. Accessed: 2026-07-28.
- Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. Evoprompt: Connecting llms with evolutionary algorithms yields powerful prompt optimizers, 2025. URL <https://arxiv.org/abs/2309.08532>.
- Seth Karten, Joel Zhang, Tersoo Upaa Jr, Ruirong Feng, Wenzhe Li, Chengshuai Shi, Chi Jin, and Kiran Vodrahalli. Continual harness: Online adaptation for self-improving foundation agents, 2026. URL <https://arxiv.org/abs/2605.09998>.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines, 2023. URL <https://arxiv.org/abs/2310.03714>.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, 2026. URL <https://arxiv.org/abs/2603.28052>.
- Xiaoxi Li, Wenxiang Jiao, Jiarui Jin, Guanting Dong, Jiajie Jin, Yinuo Wang, Hao Wang, Yutao Zhu, Ji-Rong Wen, Yuan Lu, and Zhicheng Dou. Deepagent: A general reasoning agent with scalable toolsets, 2026a. URL <https://arxiv.org/abs/2510.21618>.
- Yanzhou Li, Yiran Zhang, Xiaoyu Zhang, Xiaoxia Liu, and Yang Liu. Codeskill: Learning self-evolving skills for coding agents, 2026b. URL <https://arxiv.org/abs/2605.25430>.
- Zelong Li, Shuyuan Xu, Kai Mei, Wenyue Hua, Balaji Rama, Om Raheja, Hao Wang, He Zhu, and Yongfeng Zhang. Autoflow: Automated workflow generation for large language model agents, 2024. URL <https://arxiv.org/abs/2407.12821>.

Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. Encouraging divergent thinking in large language models through multi-agent debate, 2024. URL <https://arxiv.org/abs/2305.19118>.

Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, 2026. URL <https://arxiv.org/abs/2604.25850>.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents, 2025. URL <https://arxiv.org/abs/2308.03688>.

Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. Autoharness: improving llm agents by automatically synthesizing a code harness, 2026. URL <https://arxiv.org/abs/2603.03329>.

Zhengxi Lu, Zhiyuan Yao, Jinyang Wu, Chengcheng Han, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. Skill0: In-context agentic reinforcement learning for skill internalization, 2026. URL <https://arxiv.org/abs/2604.02268>.

Haochen Luo, Yi Huang, Sichun Luo, Fengyuan Liu, Lei Li, Zefa Hu, Junlan Feng, and Qi Liu. Harness-aware self-evolving: Co-evolving model weights, harness, and task solutions, 2026. URL <https://arxiv.org/abs/2607.03935>.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023. URL <https://arxiv.org/abs/2303.17651>.

Moonshot AI. Kimi K2.6: Advancing open-source coding. <https://www.kimi.com/blog/kimi-k2-6>, April 2026. Accessed: 2026-07-28.

Fan Nie, Lan Feng, Haotian Ye, Weixin Liang, Pan Lu, Huaxiu Yao, Alexandre Alahi, and James Zou. Weak-for-strong: Training weak meta-agent to harness strong executors, 2025. URL <https://arxiv.org/abs/2504.04785>.

OpenAI. Introducing GPT-5.5. <https://openai.com/index/introducing-gpt-5-5/>, April 2026. Accessed: 2026-07-28.

Krista Opsahl-Ong, Michael J Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs, 2024. URL <https://arxiv.org/abs/2406.11695>.

Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with "gradient descent" and beam search, 2023. URL <https://arxiv.org/abs/2305.03495>.

Qwen Team. Qwen3.5: Towards native multimodal agents. <https://qwen.ai/blog?id=qwen3.5>, February 2026. Accessed: 2026-07-28.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.

Yarui Shi, Yuxin Chen, Zhengxi Lu, Yuchun Miao, Shugui Liu, Qi GU, Xunliang Cai, Xiang Wang, and An Zhang. Skill1: Unified evolution of skill-augmented agents via reinforcement learning, 2026. URL <https://arxiv.org/abs/2605.06130>.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL <https://arxiv.org/abs/2303.11366>.

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning, 2021. URL <https://arxiv.org/abs/2010.03768>.

Yash Vishe, Rohan Surana, Xunyi Jiang, Zihan Huang, Xintong Li, Nikki Lijing Kuang, Tong Yu, Ryan A. Rossi, Jingbo Shang, Julian McAuley, and Junda Wu. Skill-r1: Agent skill evolution via reinforcement learning, 2026. URL <https://arxiv.org/abs/2605.09359>.

Xiaoxuan Wang, Haixin Wang, Alexander Taylor, Jason Cong, Yizhou Sun, and Wei Wang. Harnessbridge: Learnable bidirectional controller for llm agent harness, 2026. URL <https://arxiv.org/abs/2606.12882>.

Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xuemeng Yang, Yufan Shen, Yuxin Wang, and Botian Shi. EvolveR: Self-evolving LLM agents through an experience-driven lifecycle. In *International Conference on Machine Learning*, 2026. doi: 10.48550/arXiv.2510.16079. URL <https://arxiv.org/abs/2510.16079>.

Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning, 2026. URL <https://arxiv.org/abs/2602.08234>.

Tianshi Xu, Huifeng Wen, and Meng Li. Adapting the interface, not the model: Runtime harness adaptation for deterministic llm agents, 2026. URL <https://arxiv.org/abs/2605.22166>.

- Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers, 2024. URL <https://arxiv.org/abs/2309.03409>.
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents, 2023a. URL <https://arxiv.org/abs/2207.01206>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023b. URL <https://arxiv.org/abs/2210.03629>.
- Haiwen Yi and Xinyuan Song. Learning to control llm agent harnesses with offline reinforcement learning, 2026. URL <https://arxiv.org/abs/2607.05458>.
- Hongzhuo Yu, Fei Zhu, Guo-Sen Xie, and Ling Shao. Self-consolidation for self-evolving agents, 2026. URL <https://arxiv.org/abs/2602.01966>.
- Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. Optimizing generative ai by backpropagating language model feedback. *Nature*, 639(8055):609–616, 2025.
- Z.ai. GLM-5.2: Built for long-horizon tasks. <https://z.ai/blog/glm-5.2>, June 2026. Accessed: 2026-07-28.
- Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-harness: Harnesses that improve themselves, 2026a. URL <https://arxiv.org/abs/2606.09498>.
- Kangning Zhang, Wenxiang Jiao, Kounianhua Du, Yuan Lu, Weiwen Liu, Weinan Zhang, and Yong Yu. Looptool: Closing the data-training loop for robust llm tool calls, 2025. URL <https://arxiv.org/abs/2511.09148>.
- Mingda Zhang, Wenjin Liu, Tiesunlong Shen, Qika Lin, Rui Mao, Erik Cambria, Xiaoying Tang, and Haoran Luo. Flowsteer: Towards agents designing agentic workflows via reinforced progressive canvas editing, 2026b. URL <https://arxiv.org/abs/2602.01664>.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, 2024. URL <https://arxiv.org/abs/2308.10144>.
- Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers, 2023. URL <https://arxiv.org/abs/2211.01910>.

Appendix

A Prompts

This section presents the model-facing prompt template used to train and evaluate the harness engineer. Following the presentation style of prompt appendices, fixed instructions are shown separately from the per-example input. Angle-bracketed strings denote substituted fields rather than literal tokens. SFT and online GRPO use the same response protocol; the latter constructs the failure packet from the current frozen target-agent rollouts. The released SFT JSON contains the complete materialized messages for all three benchmarks.

A.1 Prompt for the Harness Engineer

System Instruction
You are a Harness-R1 harness engineer. Given a batch of failed <code><BENCHMARK></code> rollout traces, analyze recurring failures, then produce one reusable <code><BENCHMARK></code> code-hook harness patch as a single JSON object inside a <code><patch>...</patch></code> block.
Input
You will edit only the reusable <code><BENCHMARK></code> harness, not task answers. The chat template has already opened the assistant thinking block. Continue concise recurring-failure reasoning, close it with <code></think></code>, then output exactly one <code><patch></code> block. After your reasoning, output exactly one <code><patch></code> block containing a single JSON patch object with top-level keys <code>benchmark</code>, <code>description</code>, and <code>actions</code>. Output nothing after the <code></patch></code> block. Patch JSON top-level contract: - <code>benchmark</code>: exactly <code>"<BENCHMARK_ID>"</code> - <code>description</code>: a short, general description - <code>actions</code>: a non-empty array of <code>ADD_CODE_HOOK</code> objects <pre>ADD_CODE_HOOK ::= { "type": "add_code_hook", "hook": "on_init" "make_pre_hint" "on_before_action" "on_post_step", "code": "<PYTHON SOURCE DEFINING hook(ctx, nb)>" }</pre> Hook return contracts: - <code>on_init</code> -> <code>skills/tool_hint</code> or <code>None</code> - <code>make_pre_hint</code> -> <code>message</code> or <code>None</code> - <code>on_before_action</code> -> <code>block_and_prompt</code>, <code>rewrite_action</code>, or <code>force_action</code> - <code>on_post_step</code> -> <code>inject_hint</code> or, where supported, <code>force_action</code> General rules: - Infer reusable interventions from observable recurring failures. - Do not encode task-specific answers, indices, or identifiers. - Define <code>hook(ctx, nb)</code> without imports or global state. - Put all JSON inside the <code><patch></code> block; output nothing after <code></patch></code>. <code><BENCHMARK-SPECIFIC RUNTIME CONTEXT AND RESTRICTIONS></code> Observed no-harness rollout evidence: <code><FAILURE TRACE PACKET></code>

Figure A.1: System and input prompt template for the harness engineer. The failure packet contains a group of failed trajectories from the frozen target agent; the benchmark-specific insertion is reproduced below.

A.2 Benchmark-Specific Prompt Content

Benchmark-Specific Instruction
[WebShop] benchmark = "webshop" Active action type: add_code_hook only. Runtime context: action: tool, value, value_normalized, final_action state: page_type, clickables, remaining_steps, current_price, price_max, repeated-click/search and product-stall counters task: task_type, required_color, required_size, required_material webshop: search_queries, visited products, current product, selected attributes, available attribute options predicates: required_options_unselected, product_price_over_budget, repeated click/search, stalled product page, buy-now availability, action admissibility Use pre-action mediation only for narrow, observable mistakes such as an over-budget purchase, an unselected required option, or a repeated action. Do not hard-code product IDs, titles, ASINs, task indices, or answers. [ALFWorld] benchmark = "alfworld" Actions may contain two to four add_code_hook entries; use each hook at most once and omit hooks not supported by recurring evidence. Runtime context: action: raw, normalized, final_action, in_admissible state: repeated observation/action, invalid-action and remaining-step signals task: task_type, target_type, destination_type world: current location, inventory, object locations, visited locations, target facts, and placement facts admissible: actions currently accepted by the environment Maintain reusable task-stage state from observations and completed actions. Any exact mediated action must be selected from admissible actions. Do not copy numbered object/location instances or complete task solutions. [DBBench] benchmark = "dbbench" Actions may contain two to four add_code_hook entries; use each hook at most once and omit hooks not supported by recurring evidence. Runtime context: action: execute_sql or commit_final_answer, query, submitted answers state: SQL count/history, last result/error, error/empty/loop streaks, mutation status, candidate answer shape, remaining rounds task: task_type, answer_shape, target_table, description dbbench: SQL history, discovered columns, database response, round predicates: premature/empty commit, mutation not attempted, SQL error, empty result, unknown column, syntax error, repeated SQL, candidate answer available, remaining rounds low DBBench hooks may inject guidance or block a pending action. They do not rewrite SQL or force a commit. Do not hard-code cell values, exact answers, task indices, or ground-truth SQL.

Figure A.2: Benchmark-specific content inserted into the engineer prompt. These fields expose runtime evidence rather than hidden task answers.

A.3 Assistant Response Format

Assistant Prefill and Target
<pre><think> <concise analysis of recurring failures, the proposed reusable intervention, and the regression risk> </think> <patch> { "benchmark": "<BENCHMARK_ID>", "description": "<GENERAL PATCH DESCRIPTION>", "actions": [{ "type": "add_code_hook", "hook": "<SELECTED LIFECYCLE POSITION>", "code": "def hook(ctx, nb):\n ...\n return <STRUCTURED EFFECT>" }] } </patch></pre>

Figure A.3: Harness-engineer response format. The chat template prefills the opening `<think>` token sequence; training targets contain the remaining analysis and exactly one executable patch.

B Implementation Details

Cold-start SFT, online GRPO, and direct target-agent SFT each run on a single node with eight NVIDIA H800 GPUs. The tables below list the settings needed to reproduce each stage; framework defaults and settings that only affect memory use, such as gradient checkpointing and the ZeRO stage, are omitted.

B.1 Harness-Engineer Cold-Start SFT

GPT-5.5 generates candidate patches from failure packets in the SFT task split. We retain candidates that are executable, complete the same-batch rerun, and achieve a non-negative task-reward change. This produces approximately 1K teacher-filtered editing examples (877 in total: 381 WebShop, 248 ALFWorld, and 248 DBBench).

Parameter	Value
Base model	Qwen3.5-9B
Training examples	877
Fine-tuning type	Full parameter
Epochs	2
Context length	32,768
Global batch size	24
Optimizer	AdamW
Learning rate	10^{-5}
LR schedule	Cosine
Warmup ratio	0.03
Precision	BF16
Seed	42

B.2 Online GRPO

Parameter	Value
Candidates per prompt K	8
Rollout batch size	4 prompts
Global batch size	32 sequences
Rollout iterations per update	4
Maximum policy staleness	8
Learning rate	10^{-6}
LR schedule	Constant
Optimizer	Adam
Adam β_1, β_2	0.9, 0.98
Weight decay	0.1
GRPO clip lower / upper	0.20 / 0.28
Truncated importance sampling	Enabled; maximum weight 2.0
Entropy / explicit KL coefficient	0 / 0
Rollout temperature	0.7
Rollout top- p	0.95
Maximum prompt length	28,672
Maximum response length	12,288
Seeds (training / rollout)	1,234 / 42

B.3 Direct Target-Agent SFT

For the sequential adaptation experiment, we directly fine-tune the Qwen3.5-9B target agent on successful no-intervention trajectories from the same task-level training split used to construct the engineer data. We retain one trajectory for each benchmark and canonical task identity, yielding 2,515 complete multi-turn episodes: 901 from WebShop, 774 from ALFWorld, and 840 from DBBench. The optimization settings not listed below, including the optimizer, learning rate, schedule, warmup, precision, and seed, are identical to the cold-start SFT configuration above.

Parameter	Value
Base model	Qwen3.5-9B
Training trajectories	2,515
WebShop / ALFWorld / DBBench	901 / 774 / 840
Trajectory selection	Successful and task-deduplicated
Epochs	2
Sequence cutoff length	24,576
Global batch size	24
Target thinking	Disabled

B.4 Evaluation and Reward

Parameter	Value
Engineer reward	Full-batch mean reward change $\Delta_B(P)$
WebShop task reward	Native continuous environment reward
ALFWorld / DBBench task reward	Binary success
Invalid, no-op, or incomplete patch reward	0
Engineer temperature	0
Engineer maximum response length	12,288
Engineer thinking	Enabled, with <think> prefill
Target temperature	0
Target maximum response length	4,096
Target tool choice	auto
Target thinking	Disabled
WebShop goal seed	233

C Executable Patch Interface

A patch can intervene at four positions in the target agent’s execution lifecycle. Each hook receives benchmark-specific runtime context and returns only an effect defined by the host runtime.

Table C.1 Executable lifecycle hooks.

Hook	Invocation and effect
on_init	Before the first target decision; adds reusable task guidance or a tool hint.
make_pre_hint	Before a target decision; injects a state-conditioned message without executing an action.
on_before_action	After the target proposes an action but before environment execution; may block and reprompt or, where supported, rewrite or force the pending action.
on_post_step	After environment feedback; injects recovery guidance or, where supported, schedules a next action.

The return contract separates contextual guidance from action mediation. `make_pre_hint` returns a message, and `on_post_step` may return an `inject_hint` effect. `block_and_prompt` suppresses the current pending action and asks the frozen target to choose again. Where enabled, `rewrite_action` replaces that pending action, while `force_action` selects a concrete action for the current or next execution step. DBBench v1 accepts soft guidance and blocking but does not execute SQL rewrites or forced commits.

The engineer generates the patch before the rerun and does not participate in the subsequent task interaction. It never calls an environment tool or submits a final task answer directly; the host runtime alone interprets the hook’s structured effects. Candidates that do not yield an installable intervention, are behaviorally inert, or do not complete evaluation are treated as no intervention and receive zero reward. A valid, complete patch instead receives its full-batch performance difference, including a negative value when it causes regressions.

D Data Construction and Task Splits

We split benchmark tasks before collecting trajectories or generating harness patches. The SFT and online-RL training sets contain the task identities used to construct their respective training signals. The validation set is used only for checkpoint selection, and the test set is used only for final evaluation. Table D.1 reports numbers of distinct benchmark tasks, rather than numbers of trajectories, failure packets, generated patches, or optimizer samples.

WebShop. We use task indices 0–499 as the fixed test set. A separate training pool is randomly divided at the task-batch level into SFT and RL partitions with seed 20260603. We then reserve 100 tasks from the RL partition for validation with seed 20260623; the remaining 5,190 tasks form the RL training set. WebShop task generation uses goal seed 233 throughout baseline and patched execution.

ALFWorld. We stratify by the six ALFWorld task families. The 500-task test set contains all 109 tasks from the official `new_std` split and a stratified 391-task sample from `train_valid`. With seed 20260614, the remaining tasks are divided into 1,380 SFT tasks and an RL-side partition; 99 RL-side tasks are reserved for validation, leaving 1,280 RL training tasks.

DBBench. We shuffle the 4,803 available training tasks with seed 20260625 and assign 2,401 tasks to SFT and 2,402 to the RL side. We reserve 100 RL-side tasks for validation, leaving 2,302 RL training tasks. The separate 300-task standard test set is used only for evaluation.

Teacher filtering, failure-packet construction, benchmark balancing, and multi-candidate sampling operate within these task partitions. Their resulting record counts are therefore training-accounting quantities, not additional task splits.

Table D.1 Task-level data splits. SFT train and RL train are disjoint task partitions; Train total is their sum.

Benchmark	SFT train	RL train	Train total	Validation	Test
WebShop	5,290	5,190	10,480	100	500
ALFWorld	1,380	1,280	2,660	99	500
DBBench	2,401	2,302	4,703	100	300
Total	9,071	8,772	17,843	299	1,300

E Target-Agent Generalization

We apply the single learned editing policy to a broad set of target agents that are never used during training. For every target, the engineer reads that target’s own failure traces and generates target-specific patches; this measures transfer of the editing *policy*, not reuse of one fixed patch. Table E.1 reports the per-target, per-benchmark task success before and after installing the target-specific patches, providing the absolute levels behind the delta heatmap in the main paper. WebShop uses the fixed-seed 500-task rerun (goal seed 233); ALFWorld and DBBench use their respective test sets. The Δ Avg. column is the equal-weight average of the three benchmark deltas, and $\dagger$ marks the primary Qwen3.5-9B target that is also used in the main results.

Every target-level average is positive, and across the full 21×3 matrix 56 of 63 target–benchmark combinations improve, four are unchanged (all on WebShop), and the three small regressions are WebShop on Llama-3.1-70B (-0.4), ALFWorld on Qwen2.5-72B (-2.0), and WebShop on Gemma-4-31B-it (-0.2). The benchmark-averaged gain across the twenty unseen targets is 7.06 points, showing that a single training recipe transfers across model families and scales without any per-target retuning.

F Held-Out Task Generalization

We further test whether patches inferred from a handful of failures improve *unseen tasks*. For each benchmark and seed, every engineer observes the same ten failures from the frozen Qwen3.5-9B target, generates one benchmark-specific patch, and applies it to all remaining tasks; we repeat the protocol over three matched evidence seeds. Invalid patches are counted as no intervention (zero delta). Table F.1 reports the pooled held-out change (1,270 tasks) and, for reference, the full-split change including the ten evidence tasks (1,300 tasks); the error term is the sample standard deviation across the three seeds.

Harness-R1 improves pooled held-out success by 8.9 ± 1.5 points and is positive on every seed, whereas both frontier engineers average negative and straddle zero across seeds. The larger frontier spreads (± 2.5 and ± 3.6) reflect swings between marginal gains and sizable regressions, so converting sparse failures into a broadly useful edit is a capability that scale alone does not confer.

G Lifecycle-Position Ablation

To attribute the improvement to specific intervention points, we hold the frozen vanilla target and the generated patches fixed and disable one lifecycle position at a time, alongside a no-intervention control and the full patch. Each configuration reruns the target three times per benchmark, and we report the equal-benchmark-weighted success rate used in the main results. Table G.1 lists the per-benchmark and averaged success behind the ablation figure in the main paper.

The full patch reaches 53.1% average success, 8.9 points above no intervention. Removing pre-action mediation or post-feedback recovery causes the largest drops (3.9 and 3.3 points), while removing episode initialization or pre-decision costs only 0.9 and 0.6 points. The dominant position is environment-dependent: pre-action mediation matters most on WebShop (success falls from 41.6 to 31.5), whereas post-feedback recovery matters most on ALFWorld (52.1 to 41.9). Because a single patch can coordinate several positions, these effects are conditional and should not be summed into a universal importance ranking.

Table E.1 Target-agent generalization: per-benchmark task success rate before and after installing target-specific patches (%), with the equal-weight benchmark average of the deltas (Δ Avg., pp). Rows are grouped by model family; $\dagger$ marks the primary Qwen3.5-9B target. WebShop uses the fixed-seed 500-task rerun; ALFWorld and DBBench use their test sets.

Target agent	WebShop		ALFWorld		DBBench		Δ Avg.
	Before	After	Before	After	Before	After	
Llama-3.1-8B	21.2	31.0	2.0	10.0	16.7	30.3	+10.5
Llama-3.1-70B	39.2	38.8	21.0	35.2	31.7	33.7	+5.3
Llama-3.2-1B	0.0	0.0	0.0	0.2	8.0	14.3	+2.2
Llama-3.2-3B	8.4	13.6	1.2	7.2	7.0	16.7	+7.0
Llama-3.3-70B	35.4	41.8	27.4	46.2	60.3	63.0	+9.3
Gemma-3-1B	0.0	0.0	0.0	1.2	0.3	2.7	+1.2
Gemma-3-4B	13.8	28.4	4.0	10.4	14.3	29.3	+12.0
Gemma-3-12B	22.8	32.2	12.8	18.8	41.0	56.7	+10.4
Gemma-3-27B	27.8	37.2	18.6	29.4	52.3	60.0	+9.3
Gemma-4-12B-it	39.4	39.4	35.8	55.2	61.3	67.7	+8.6
Gemma-4-26B-A4B-it	39.2	39.6	49.2	65.4	60.0	69.0	+8.5
Gemma-4-31B-it	42.0	41.8	54.4	73.2	65.7	69.0	+7.3
Qwen2.5-72B	37.8	39.6	70.8	68.8	51.3	54.7	+1.0
Qwen3-4B	25.0	38.6	22.8	29.1	38.7	55.7	+12.3
Qwen3-8B	30.6	34.6	23.0	27.7	49.3	57.3	+5.6
Qwen3-14B	33.8	37.2	22.3	39.0	50.0	60.0	+10.0
Qwen3.5-4B	33.2	36.8	20.7	38.6	60.7	65.0	+8.6
Qwen3.5-9B$\dagger$	31.2	42.2	40.6	53.2	61.0	65.3	+9.3
Qwen3.5-27B	42.0	42.0	72.4	81.3	70.3	72.3	+3.6
Qwen3.5-35B-A3B	30.6	31.8	62.2	69.2	62.7	68.7	+4.7
Qwen3.6-27B	43.4	44.2	70.6	78.6	69.7	72.7	+3.9
Mean, 20 unseen targets	28.3	32.4	29.4	39.1	43.6	50.9	+7.1
Mean, all 21 targets	28.4	32.9	30.0	39.8	44.4	51.6	+7.2

Table F.1 Held-out-task generalization from sparse failure evidence (Δ success, pp; mean $\pm$ sample std over three seeds). Valid counts the seed-benchmark patches that installed a real intervention out of nine.

Engineer	Valid	Held-out (1,270)	Full split (1,300)
Harness-R1	9/9	+8.9$\pm$1.5	+9.2$\pm$1.5
Qwen3.5-397B	8/9	−4.3 $\pm$ 2.5	−3.9 $\pm$ 2.5
DeepSeek-V4-Pro	6/9	−0.4 $\pm$ 3.6	−0.2 $\pm$ 3.5

Table G.1 Fixed-patch lifecycle-position ablation on the vanilla target (success rate, %). The **Avg.** column is the equal-weight benchmark average; parenthesized values are the drop relative to the full patch.

Configuration	WebShop	ALFWorld	DBBench	Avg.
Full patch	41.6	52.1	65.4	53.1
w/o episode init	41.5	51.3	63.8	52.2 (−0.9)
w/o pre-decision	41.5	50.4	65.6	52.5 (−0.6)
w/o pre-action	31.5	50.7	65.4	49.2 (−3.9)
w/o post-feedback	41.7	41.9	65.8	49.8 (−3.3)
No intervention	31.8	40.7	60.1	44.2

H Qualitative Case Studies

We examine three stored evaluations from the validation-selected Harness-R1 engineer used for the main frozen-target results. In every case, the target agent is the same frozen Qwen3.5-9B model before and after patch installation, and the patched run uses the same ten tasks as its baseline evidence. We inspect the runtime trace in addition to the generated code, so that an intended edit is not mistaken for an intervention that actually executed. These cases illustrate distinct mechanisms and limitations; aggregate claims are based on the full evaluations in the main paper rather than on the selected examples.

H.1 WebShop: Correcting a Premature Purchase

Observed failure and generated edit. In WebShop batch 008, several trajectories reached a relevant, in-budget product but issued Buy Now before choosing an option required by the instruction. The engineer generated a single pre-action intervention. It activates only for a normalized Buy Now action and blocks the action when either the current price exceeds the budget or a required product option remains unselected. The resulting message asks the target to choose a visible matching option, or return to search if no such option exists.

Task-level behavior. One task requests a synthetic hairpiece with a black-brown color and a price below \$40. The baseline target finds a suitable product but purchases it without selecting the color, receiving a partial reward of 0.667. With the patch installed, the target initially proposes the same premature purchase. The guard blocks it, after which the target selects black brown and then purchases the item, receiving reward 1.0.

Run	Relevant action sequence	Reward
No intervention	Search → open product → Buy Now with color unselected	0.667
Harness-R1	Search → open product → attempted Buy Now → guard message → select black brown → Buy Now	1.000

Across the ten-task batch, the same guard raises full successes from 2 to 5 and mean WebShop reward from 0.682 to 0.768, while preserving both tasks that were already fully successful. This case shows that an effective harness edit need not replace the target’s policy with a large controller: a low-bandwidth intervention at the point of an unsafe action can preserve the target’s search behavior while changing the final outcome.

H.2 ALFWorld: Coordinating Multiple Lifecycle Positions

Observed failure and generated edit. ALFWorld batch 045 contains recurrent failures in which the target finds an object but omits a required transformation, moves toward the wrong receptacle, or enters a transform-place loop. The generated patch coordinates four positions in the execution lifecycle:

Lifecycle position	Installed behavior
Episode initialization	Initialize the current stage and provide the reusable find-take-transform-place ordering.
After environment feedback	Update whether the target is held, the required transformation is complete, and the destination has been reached.
Before model decision	Inject a stage-specific hint for the next unresolved subgoal.
Before environment execution	Block a premature placement or a placement at the wrong destination.

Runtime behavior. The intervention trace records 56 stage hints or guard messages across the batch. For a task requiring a hot mug in a cabinet, the target successively receives guidance to take the mug, heat it using the microwave, navigate to the cabinet, and place the mug. For a task requiring a cooled egg on a countertop, the target attempts to place the egg on a dining table; the pre-action guard rejects that destination, and the subsequent trajectory places the egg on the requested countertop.

Table H.1 Overview of the qualitative cases.

Environment	Success (before)	Success (after)	Δ	Primary behavior illustrated
WebShop	2/10	5/10	+3	A narrow guard delays purchase until required options are selected.
ALFWorld	1/10	6/10	+5	State tracking, stage-specific guidance, and an action guard form a closed loop.
DBBench	4/10	6/10	+2	Schema recovery and format-preserving mutation outperform a valid GLM-5.2 edit.

Example	Effective intervention sequence	Outcome
Hot mug in cabinet	Take-target hint $\rightarrow$ heat-at-microwave hint $\rightarrow$ go-to-destination hint $\rightarrow$ put-target hint	Failure $\rightarrow$ success
Cooled egg on countertop	Stage hint $\rightarrow$ attempted wrong placement $\rightarrow$ destination guard $\rightarrow$ corrected placement	Failure $\rightarrow$ success

At batch level, the patch rescues six baseline failures but regresses one baseline success, producing a net change from 1/10 to 6/10. It also fails to resolve every task: one two-object trajectory continues to alternate between destination and placement guidance. The example therefore demonstrates a genuine closed-loop harness policy, while also showing that stage tracking can remain imperfect.

H.3 A Failure Case of Direct Harness Editing

An off-the-shelf model can access the complete lifecycle interface yet still produce harmful interventions. On ALFWorld, Gemini-3.5-Flash receives full failure evidence and may edit all four lifecycle positions. Its patches reduce success from 208/500 (41.6%) to 177/500 (35.4%), a drop of 6.2 percentage points. Among 39 valid patches, 21 reduce batch success, 12 improve it, and 6 leave it unchanged.

Overgeneralized action intervention. The largest regression occurs in a batch whose success falls from 7/10 to 0/10. The patch installs broad `on_before_action` rules that force actions from a locally plausible stage estimate. On two-object tasks, it prematurely places the first object rather than collecting both objects before placement, overriding decisions that the frozen target agent previously executed correctly. This case illustrates why execution traces and a powerful base model alone do not yield a reliable harness editor: a plausible diagnosis can still compile into overly aggressive runtime behavior. Harness-R1 instead post-trains the editing policy on realized task outcomes, directly penalizing patches that degrade rerun performance.

H.4 DBBench: Preserving Schema and Stored-Value Conventions

Observed failure and generated edit. DBBench batch 022 contains recurring failures around multi-word identifiers, schema recovery, and exact mutation values. Harness-R1 generates a stage-aware patch that recommends schema inspection after identifier errors, asks the target to inspect the affected row before mutation, and verifies the stored value before commit. The patch raises the frozen target from 4/10 to 6/10. On the same baseline evidence and tasks, a valid GLM-5.2 patch raises the result only to 5/10.

Task-level contrast. One task asks the agent to change the length of the Moosehead Grand Prix entry in the multi-word table Race Schedule. The no-intervention trajectory recovers the quoted table name and observes the existing value 3 Hours, but writes 4 hours; the exact-format evaluator marks the task incorrect. GLM-5.2 supplies general backtick and mutation-verification guidance, yet its guided trajectory makes the same lower-case write. Harness-R1 first triggers schema recovery, inspects the existing row, writes 4 Hours to match the stored convention, and verifies the row before committing.

Runtime condition	Batch success	Example outcome
No intervention	4/10	4 hours (failure)
GLM-5.2 patch	5/10	4 hours (failure)
Harness-R1 patch	6/10	4 Hours (success)

This paired example does not rely on an invalid competitor output: both engineers produce executable patches. The difference is that the Harness-R1-guided run converts schema and row evidence into the exact stored representation required by the task.

H.5 Cross-Case Interpretation

WebShop. The recurring failure is premature purchase. Harness-R1 installs a narrow action guard conditioned on runtime predicates, although the guard cannot repair an earlier choice of the wrong product.

ALFWorld. The recurring failures are omitted transformations and incorrect placement. Harness-R1 combines persistent stage state, targeted hints, and a placement guard, while routing and two-object state can still cause regressions or loops.

DBBench. The recurring failures involve multi-word identifiers and exact mutation formats. Harness-R1 uses schema recovery, row inspection, and format-preserving verification; some value conventions still require stronger neighborhood-level checks.

Together, the cases show that Harness-R1 learns environment-dependent editing policies rather than one fixed prompt. Outcome-grounded post-training increases useful executable edits without guaranteeing complete or regression-free rules.